

Team Name

LEGAL VISIONARIES

Project Title

LegalEase – AI-Powered Legal Document Generator

Team Members

1. **Rithika**
2. **Sindhuja**
3. **Sumathi**
4. **Sathvika**

PHASE 2: REQUIREMENT ANALYSIS

1. Problem Statement

Creating legal documents manually can be time-consuming and difficult for users who are unfamiliar with legal terminology, document structure, and professional formatting.

Users may need different types of legal documents such as agreements, contracts, Non-Disclosure Agreements, lease agreements, and employment documents. Preparing these documents repeatedly and formatting them professionally can require additional effort.

LegalEase is proposed as an AI-powered legal document generation platform that assists users in creating structured and editable legal document drafts from simple user-provided information.

2. Project Objectives

The main objectives of LegalEase are:

1. To provide an easy-to-use interface for generating legal document drafts.
2. To generate structured document content using Generative AI.
3. To allow users to provide document type, parties, terms and conditions, and effective date.
4. To provide an editable preview of the generated document.
5. To support multiple document types.
6. To allow documents to be exported in TXT, DOCX, and PDF formats.
7. To provide professional document formatting.
8. To simplify the initial drafting process for non-technical users.

3. Functional Requirements

The system shall provide the following functions:

3.1 User Input

The system shall allow users to enter:

- Document type
- Parties involved
- Terms and conditions

- Effective date

3.2 Document Generation

The system shall process the user's information and generate a structured legal document using Generative AI.

3.3 Document Preview

The generated document shall be displayed to the user for review.

3.4 Document Editing

Users shall be able to modify the generated document before downloading it.

3.5 Document Export

The system shall allow users to download the generated document in:

- TXT
- DOCX
- PDF

3.6 Multiple Document Types

The system shall support document types such as:

- Contracts
- Agreements
- NDA
- Lease Agreement
- Employment documents

4. Non-Functional Requirements

4.1 Usability

The application should provide a simple and user-friendly interface.

4.2 Performance

The system should process user requests and generate documents within a reasonable amount of time.

4.3 Reliability

The system should provide consistent output for valid user inputs.

4.4 Maintainability

The application should have a structured architecture so that features can be modified or extended easily.

4.5 Security

User-provided information and API credentials should be handled securely.

4.6 Portability

The generated documents should be available in commonly used formats such as TXT, DOCX, and PDF.

5. Hardware Requirements

The project can be developed and tested using a standard computer.

Minimum Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Minimum 10 GB available storage
- Internet connection
- Keyboard and mouse

6. Software Requirements

The project requires the following software and technologies:

- Python 3.10+
- FastAPI
- Uvicorn
- Streamlit
- Google Gemini API
- Python-dotenv
- python-docx
- FPDF
- Pillow
- Requests
- Git
- GitHub

- Web browser
- Visual Studio Code or another Python-compatible IDE

7. User Requirements

The user should be able to:

1. Select the required document type.
2. Enter the required party information.
3. Enter terms and conditions.
4. Enter the effective date.
5. Generate a legal document draft.
6. Preview the generated content.
7. Edit the generated content.
8. Download the final document.

8. Input Requirements

The system requires the following inputs:

Input	Description
Document Type	Type of legal document required
Parties	Names/details of parties involved
Terms	Terms and conditions to include
Effective Date	Date from which the document becomes effective

9. Output Requirements

The system shall generate a structured legal document based on the provided inputs.

The output shall:

- Contain appropriate headings and sections.
- Include the provided party information.
- Include the specified terms and conditions.
- Include the effective date.
- Be available for preview and editing.

- Be downloadable as TXT, DOCX, or PDF.

10. System Architecture Requirements

The initial system architecture shall consist of:

User

↓

Streamlit Frontend

↓

FastAPI Backend

↓

Gemini AI

↓

Generated Legal Document

↓

Editable Preview

↓

TXT / DOCX / PDF

The frontend will provide the user interface, while the FastAPI backend will process requests and communicate with the AI generation component.

11. System Constraints

The project may have the following constraints:

1. An internet connection is required for AI-based document generation.
2. AI-generated content may require user review and editing.
3. API access and API credentials are required for AI integration.
4. The system depends on the availability of the selected AI service.
5. The project focuses on assisting with initial document drafting rather than replacing professional legal advice.

12. Assumptions

The project assumes that:

1. Users provide accurate information.

2. Users review generated documents before using them.
3. The required AI API is available.
4. Users have access to an internet connection.
5. The development environment has the required Python packages installed.

13. Requirement Summary

The requirement analysis establishes the basic requirements for developing LegalEase.

The system will accept document-related information from users, process the information through the backend and Generative AI system, generate a structured legal document, provide an editable preview, and allow the final document to be downloaded in multiple formats.

These requirements will serve as the foundation for the **Project Design, Project Planning, and Development phases**.